

Звіт

до Лабораторної роботи №14

Базова багатопотоковість у Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є набуття практичних навичок базового багатопотокового програмування в Java.

У роботі потрібно показати створення потоків, запуск паралельних дій, використання `Thread.sleep()`, очікування завершення потоків через `join()` та пояснення недетермінованого порядку виконання потоків.

2. Варіант роботи

Варіант 5 — медіатека

Для обраного варіанта було реалізовано консольну Java-програму для роботи з медіатекою. У програмі кілька незалежних завдань виконуються у власних потоках.

3. Опис предметної області

У лабораторній роботі реалізовано просту медіатеку.

Медіатека містить список фільмів і список подкастів. Над цими даними виконуються незалежні дії, які можуть працювати паралельно.

У проєкті створено такі класи предметної області:

- Movie — фільм;
- Podcast — подкаст;
- MediaCatalog — каталог медіатеки.

Також створено сервісний клас `MediaService`, який містить логіку обробки даних медіатеки.

4. Класи предметної області

4.1 Клас `Movie`

Клас `Movie` описує фільм.

Поля класу:

- `title` — назва фільму;
- `genre` — жанр;
- `rating` — рейтинг;
- `durationMinutes` — тривалість фільму у хвиликах.

Цей клас використовується під час сортування фільмів і формування рекомендацій.

4.2 Клас `Podcast`

Клас `Podcast` описує подкаст.

Поля класу:

- `title` — назва подкасту;
- `host` — ведучий;
- `episodesCount` — кількість епізодів.

Цей клас використовується як частина загального медіакаталогу.

4.3 Клас `MediaCatalog`

Клас `MediaCatalog` зберігає дані медіатеки.

Він містить:

```
private final List<Movie> movies;
```

```
private final List<Podcast> podcasts;
```

Клас має методи для отримання списку фільмів, списку подкастів і загальної кількості елементів у каталозі.

5. Клас `MediaService`

Клас `MediaService` містить основну логіку обробки медіатеки.

У ньому реалізовано методи:

- `scanCatalog()` — сканування каталогу;
- `sortMoviesByRating()` — сортування фільмів за рейтингом;
- `recommendTopMovies()` — формування списку рекомендованих фільмів;
- `calculateAverageRating()` — обчислення середнього рейтингу фільмів.

Цей клас не залежить від потоків, тому його логіку можна тестувати окремо.

6. Багатопотоковий сценарій

У роботі реалізовано три незалежні завдання:

- сканування медіакаталогу;
- сортування фільмів за рейтингом;
- генерація рекомендацій.

Для цього створено такі класи:

- `CatalogScanThread`;
- `SortTask`;
- `RecommendationTask`.

Клас `CatalogScanThread` наслідує `Thread`. Класи `SortTask` і `RecommendationTask` реалізують інтерфейс `Runnable`.

7. Потік через `extends Thread`

Клас `CatalogScanThread` є прикладом створення потоку через наслідування від `Thread`.

Він відповідає за сканування медіакаталогу.

У методі `run()` цей потік:

- виводить ім'я поточного потоку;
- імітує поетапну роботу через Thread.sleep();
- викликає метод scanCatalog() сервісу MediaService;
- зберігає результат сканування.

Приклад створення потоку:

```
CatalogScanThread scanThread = new CatalogScanThread(catalog, service);
```

8. Завдання через implements Runnable

У роботі реалізовано два завдання через інтерфейс Runnable.

Клас SortTask відповідає за сортування фільмів за рейтингом.

```
SortTask sortTask = new SortTask(catalog, service);
```

```
Thread sortThread = new Thread(sortTask, "SortThread");
```

Клас RecommendationTask відповідає за формування списку рекомендованих фільмів.

```
RecommendationTask recommendationTask = new  
RecommendationTask(catalog, service);
```

```
Thread recommendationThread = new Thread(recommendationTask,  
"RecommendationThread");
```

Такий підхід відокремлює саму задачу від механізму запуску потоку.

9. Запуск потоків

Усі робочі потоки запускаються методом start():

```
scanThread.start();
```

```
sortThread.start();
```

```
recommendationThread.start();
```

Метод start() створює новий потік виконання.

Якщо викликати run() напямую, код буде виконуватись у поточному потоці main, тому це не буде реальною багатопотоковістю.

10. Очікування завершення потоків

Після запуску потоків використовується метод join():

```
scanThread.join();
```

```
sortThread.join();
```

```
recommendationThread.join();
```

Метод `join()` потрібен для того, щоб головний потік `main` дочекався завершення всіх робочих потоків.

Тільки після цього програма виводить фінальні результати:

- результат сканування каталогу;
- відсортовані фільми;
- рекомендовані фільми;
- середній рейтинг фільмів.

11. Використання `sleep()`

У класах потоків використано `Thread.sleep()`.

Цей метод імітує тривалу або поетапну роботу.

Наприклад, у `CatalogScanThread` сканування каталогу виконується в декілька кроків. Через це можна побачити, що потоки працюють незалежно один від одного, а порядок повідомлень у консолі може змінюватися.

12. Імена потоків

У роботі використовуються власні імена потоків:

- `CatalogScanThread`;
- `SortThread`;
- `RecommendationThread`.

У методах `run()` використовується:

```
Thread.currentThread().getName()
```

Це дозволяє бачити в консолі, який саме потік виконує певну дію.

13. Недетермінований порядок виконання

Порядок виконання потоків не гарантується.

Навіть якщо потоки запускаються в одному порядку, повідомлення в консолі можуть з'являтися в іншому порядку.

Це нормальна властивість багатопотокових програм. На порядок виконання впливає планувальник потоків операційної системи.

Тому в програмі правильність роботи не залежить від конкретного порядку проміжних повідомлень.

14. UML-діаграми

Для проєкту було створено UML-діаграму класів та діаграму послідовності. Вони показують структуру програми та сценарій запуску потоків.

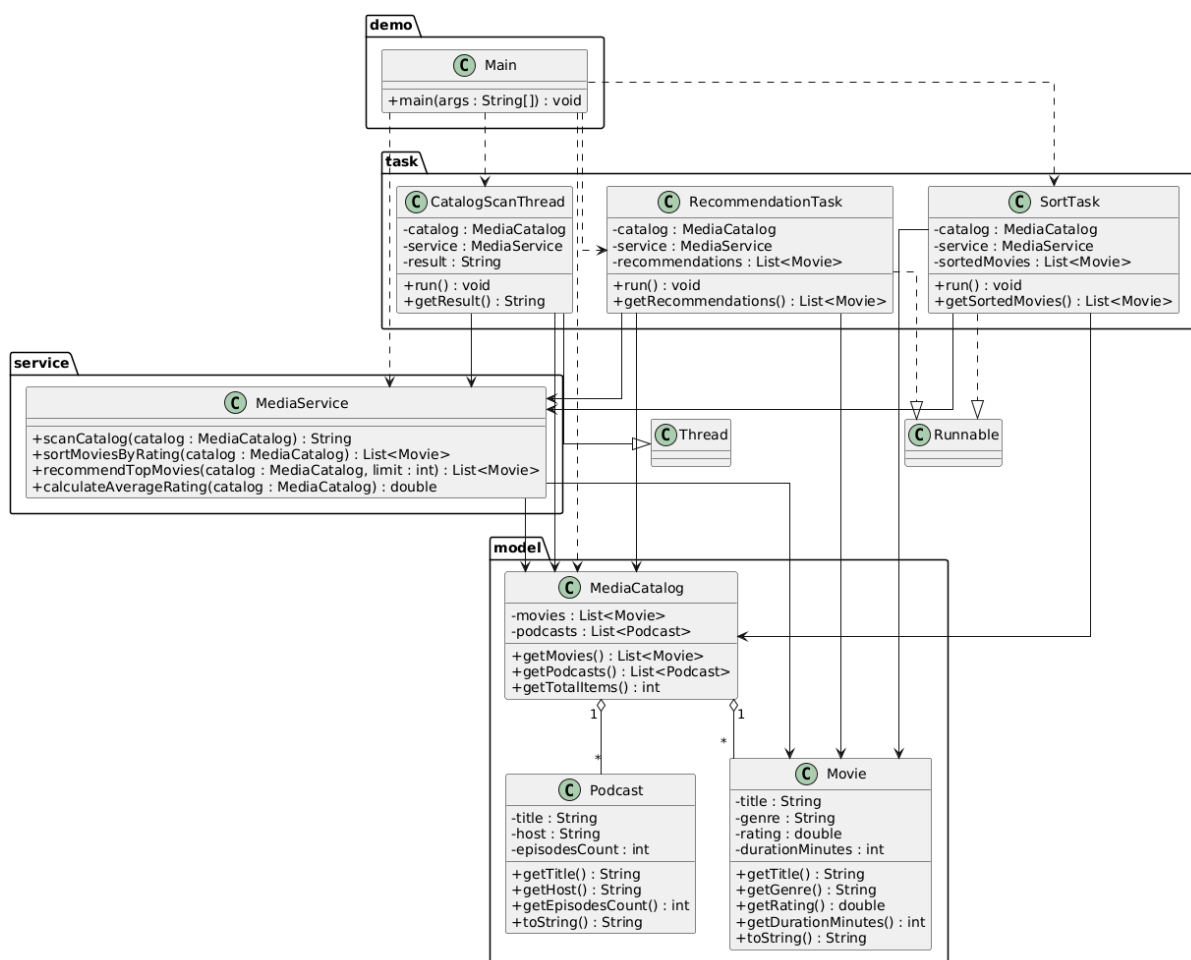


Рисунок 1 — UML-діаграма класів лабораторної роботи

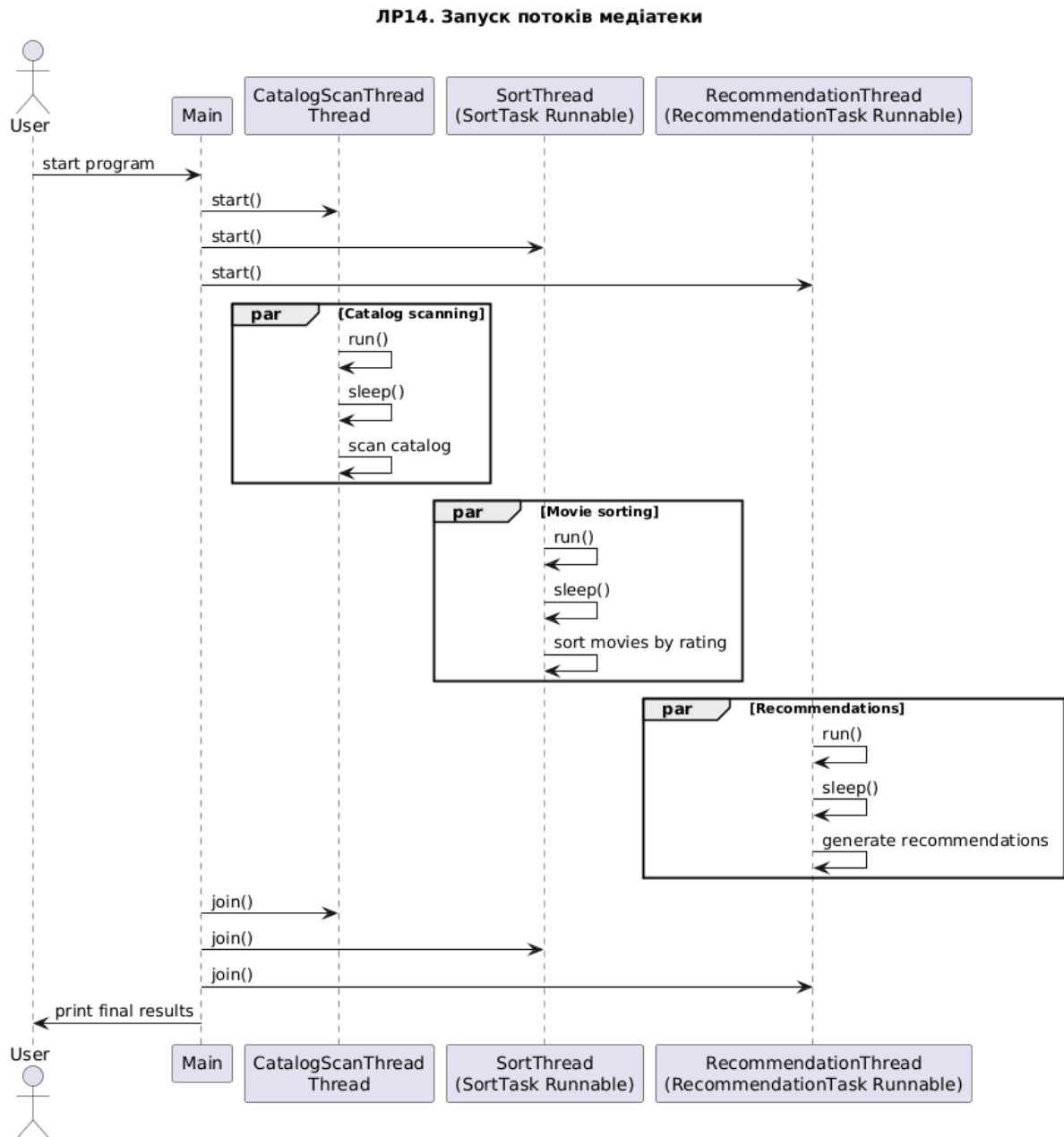


Рисунок 2 — Діаграма послідовності запуску потоків

15. Демонстрація роботи програми

У класі Main створюються:

- список фільмів;
- список подкастів;
- об'єкт MediaCatalog;
- об'єкт MediaService;
- три робочі потоки.

Після запуску потоків програма чекає їх завершення через `join()` і виводить результати роботи.

```
Запуск потоків медіатеки
SortThread: початок сортування фільмів
CatalogScanThread: початок сканування каталогу
RecommendationThread: початок генерації рекомендацій
CatalogScanThread: крок сканування 1
CatalogScanThread: крок сканування 2
SortThread: сортування завершено
RecommendationThread: рекомендації сформовано
CatalogScanThread: крок сканування 3
CatalogScanThread: сканування завершено

Усі потоки завершили роботу

Результат сканування:
У каталозі знайдено: 5 фільмів, 2 подкастів, усього: 7

Фільми після сортування за рейтингом:
Фільм: Темний лицар, жанр: Бойовик, рейтинг: 9.0, тривалість: 152 хв
Фільм: Початок, жанр: Фантастика, рейтинг: 8.8, тривалість: 148 хв
Фільм: Інтерстеллар, жанр: Фантастика, рейтинг: 8.7, тривалість: 169 хв
Фільм: Матриця, жанр: Фантастика, рейтинг: 8.7, тривалість: 136 хв
Фільм: Джокер, жанр: Драма, рейтинг: 8.4, тривалість: 122 хв

Рекомендовані фільми:
Фільм: Темний лицар, жанр: Бойовик, рейтинг: 9.0, тривалість: 152 хв
Фільм: Початок, жанр: Фантастика, рейтинг: 8.8, тривалість: 148 хв
Фільм: Інтерстеллар, жанр: Фантастика, рейтинг: 8.7, тривалість: 169 хв

Середній рейтинг фільмів:
8.72
```

Рисунок 3 — Результат запуску програми

16. Перевірка багатопотоковості

Для перевірки багатопотоковості в дебагері було поставлено breakpoint після запуску потоків через `start()`.

Стан потоків перевірявся через `getState()`:

```
"scan=" + scanThread.getState()
    + ", sort=" + sortThread.getState()
    + ", recommendation=" + recommendationThread.getState()
```

Було отримано результат:

`scan=RUNNABLE, sort=RUNNABLE, recommendation=RUNNABLE`

Це означає, що всі три потоки були запущені й передані JVM для виконання.

Також можливий стан `TIMED_WAITING`, якщо потік у цей момент перебуває в `Thread.sleep()`. Це також підтверджує, що потік уже працює.

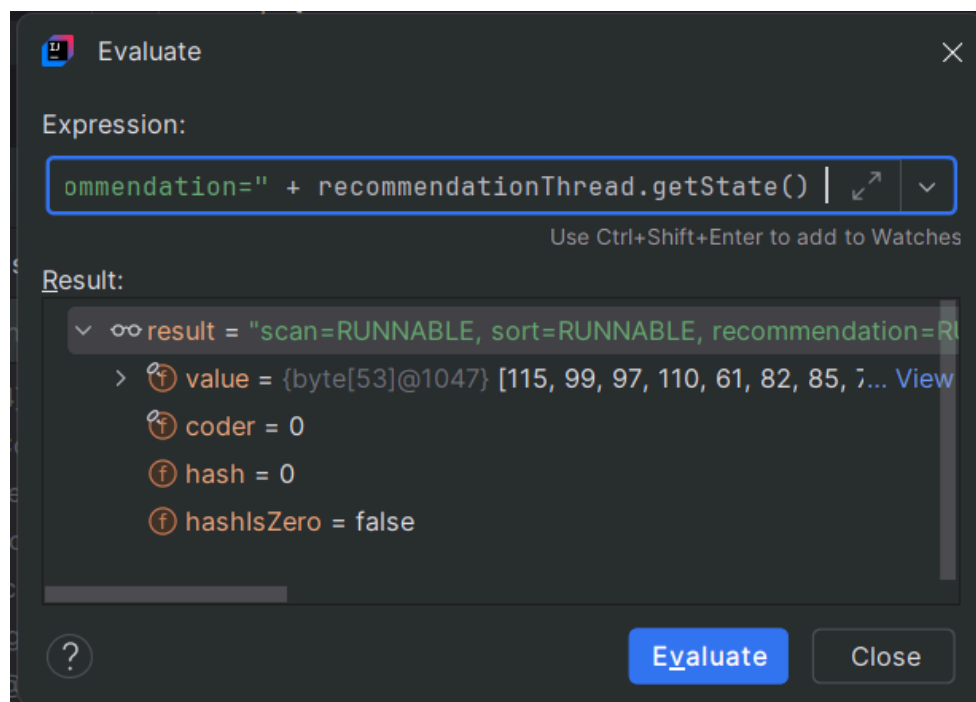


Рисунок 4 — Перевірка станів потоків у дебагері

17. Тестування

У роботі реалізовано модульні тести JUnit Jupiter.

Тестові класи:

- `MediaServiceTest`;
- `RecommendationTaskTest`.

Тести перевіряють:

- сортування фільмів за рейтингом;
- обчислення середнього рейтингу;
- формування списку рекомендованих фільмів;
- роботу RecommendationTask при прямому виклику run().

У тестах не перевіряється точний порядок виконання потоків, тому що він залежить від планувальника потоків і може змінюватися при кожному запуску.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ua.khpi.oop.lab14.service.MediaServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.049 s -- in ua.khpi.oop.lab14.service.MediaServiceTest
[INFO] Running ua.khpi.oop.lab14.task.RecommendationTaskTest
main: яюѳрѳюъ ухэхѳрѳ|ѳ ѳхьюѳхѳрѳѳ|щ
main: ѳхьюѳхѳрѳѳ|ѳ ѳѳюѳѳѳрѳѳ
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.314 s -- in ua.khpi.oop.lab14.task.RecommendationTaskTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 5 — Результат виконання модульних тестів

18. Висновок

У ході виконання лабораторної роботи було реалізовано просту медіатеку з використанням базової багатопотоковості Java.

Було створено класи предметної області Movie, Podcast і MediaCatalog, сервіс MediaService, потік CatalogScanThread та завдання SortTask і RecommendationTask.

У програмі показано два способи створення потоків: через наслідування від Thread і через реалізацію Runnable.

Усі потоки запускаються методом start(), виконують незалежні завдання, імітують поетапну роботу через Thread.sleep() та завершуються з очікуванням через join().

Детермінована логіка винесена в сервіс MediaService, тому її можна тестувати окремо від планувальника потоків.

Отже, лабораторна робота демонструє базові принципи багатопотоковості Java без складної синхронізації спільного змінюваного стану.